

GATE 2027

Computer Science and Information Technology (CS)

In-depth guide: subject-by-subject concepts, formulas, question patterns, traps, a 19-week schedule and exam strategy.

Contents	
1	Paper snapshot and how marks are earned
2	Subject priority map and study order
3	Engineering Mathematics (Discrete, Linear Algebra, Calculus, Probability)
4	Digital Logic
5	Computer Organization and Architecture
6	Programming and Data Structures
7	Algorithms
8	Theory of Computation
9	Compiler Design
10	Operating Systems
11	Databases
12	Computer Networks
13	General Aptitude (15 marks)
14	19-week schedule (28 Sep 2026 - 6 Feb 2027)
15	PYQs, mocks, error log and revision system
16	Exam-day strategy, resources, sources

Verify officially. Weightage figures are planning estimates. The official CS syllabus PDF is on gate2027.iitm.ac.in; your exam date, session and city come only with the admit card.

1. Paper snapshot and how marks are earned

Feature	Detail
Exam days (GATE 2027)	6, 7, 13, 14, 20, 21 February 2027; results expected about 19 March 2027
Format	Online CBT, 3 hours, 65 questions, 100 marks; virtual calculator provided
Split	General Aptitude 10 Q / 15 marks; CS core 55 Q / 85 marks
Question types	MCQ (one correct), MSQ (one or more correct, no partial credit), NAT (typed number)
Negative marking	Wrong MCQ only: minus 1/3 (1-mark), minus 2/3 (2-mark). None for MSQ and NAT
Score validity	3 years from result date

Where the marks come from

- **Never-skip marks:** General Aptitude (15) and Engineering Maths (about 10 to 15). Together about a quarter of the paper, and highly learnable.
- **Large scoring blocks:** Algorithms, Programming and Data Structures, Operating Systems, Databases, Computer Networks, Computer Organization each contribute roughly 6 to 10 marks.
- **Steady mid-size blocks:** Theory of Computation, Digital Logic, Compiler Design (about 4 to 8 marks each). Compiler and Digital reward fast, mechanical practice.
- **Question mix:** a large share of questions are NAT/calculation style (cache, pipelining, scheduling, subnetting, probability, graph counting). Speed on numerical problems separates ranks.

A note on expected-value guessing

A blind guess on a 4-option MCQ has expected value zero. Once one option is eliminated the guess is positive. NAT and MSQ have no penalty, so never leave them blank - but MSQ needs every correct option and no wrong one, so choose only options you can defend.

2. Subject priority map and study order

Tier	Subjects	Why
A - study first	Engineering Maths, Algorithms, Programming and Data Structures, Operating Systems	Highest marks, and Maths/Algorithms are also the base for other subjects (and the DA paper)
B - core scorers	Databases, Computer Networks, Computer Organization	Consistent marks; calculation-heavy and pattern-driven
C - quick wins	Digital Logic, Compiler Design	Small syllabus, mechanical problems - fast to master, easy to lose marks on carelessness
D - needs conceptual work	Theory of Computation	Abstract; most candidates need fresh study, but the question patterns repeat heavily

Suggested order: Maths first (shared with DA), then Algorithms and PDS, then OS, DBMS, CN, COA, then Digital, Compiler, TOC. Digital Logic before COA; Discrete Maths before TOC and Algorithms.

3. Engineering Mathematics

Approx. weightage: 10-15 marks (Discrete ~5-8, Linear Algebra ~2-3, Calculus ~2, Probability and Statistics ~3-4)

Study Discrete Maths, Linear Algebra and Probability to the depth needed for DA as well - it is one investment for two papers.

3A. Discrete Mathematics

- **Propositional and first-order logic:** connectives, equivalences, tautology and contradiction, normal forms, inference rules; quantifier order, negation of quantified statements, translating English to logic.
- **Sets, relations, functions:** reflexive, symmetric, antisymmetric, transitive; equivalence relations and partitions; partial orders, Hasse diagrams, maximal/minimal elements, chains and antichains; injective, surjective, bijective, composition, inverse.
- **Lattices:** lub/glb, distributive and complemented lattices, Boolean algebra as a lattice.
- **Groups:** closure, associativity, identity, inverse; subgroups, cyclic groups, order of an element, Lagrange's theorem, abelian groups; Z_n under addition; $U(n)$ under multiplication.
- **Graph theory:** handshake lemma, degree sequences, trees, connectivity, Euler and Hamiltonian paths, planarity, colouring, matching, independent set, clique, cut vertex/edge, bipartite graphs.
- **Combinatorics:** permutations and combinations, stars and bars, inclusion-exclusion, derangements, pigeonhole, recurrence relations (characteristic equation), generating functions.

Item	Fact
Relations on an n-element set	All: 2^{n^2} . Reflexive: $2^{n^2 - n}$. Symmetric: $2^{n(n+1)/2}$. Antisymmetric: $2^n \times 3^{n(n-1)/2}$. Equivalence relations = Bell number B_n
Functions A to B, A =m, B =n	All: n^m . One-to-one: $n!/(n-m)!$ ($m \leq n$). Onto: by inclusion-exclusion, sum of $(-1)^k C(n,k)(n-k)^m$
Graph identities	Sum of degrees = $2e$. Complete graph K_n has $n(n-1)/2$ edges. Tree has $n-1$ edges. Labelled trees on n vertices: n^{n-2} (Cayley)
Planar graphs	Euler: $v - e + f = 2$ (connected). Simple planar: $e \leq 3v - 6$; if triangle-free $e \leq 2v - 4$. K_5 and $K_{3,3}$ are non-planar. Four-colour theorem
Euler path/circuit	Circuit: connected and all degrees even. Path: exactly 0 or 2 odd-degree vertices
Counting	Stars and bars: non-negative solutions of $x_1 + \dots + x_k = n$ is $C(n+k-1, k-1)$. Derangements: $D_n = n! \times \sum (-1)^i / i!$ ($D_3=2, D_4=9$)
Recurrences	Solve $a_n = c_1 a_{n-1} + c_2 a_{n-2}$ via $x^2 - c_1 x - c_2 = 0$; repeated root r gives $(A + Bn)r^n$
Groups	Order of subgroup divides order of group. Every group of prime order is cyclic. In cyclic group of order n , number of generators is $\phi(n)$

- **Patterns:** count relations/functions; number of edges/regions/components; chromatic number; Hasse diagram properties; logic equivalences; number of solutions of a recurrence; group tests (is it a group?); pigeonhole application.
- **Traps:** antisymmetric is not the opposite of symmetric; a relation can be both. Handshake lemma applies to simple graphs and multigraphs (loops count 2).

3B. Linear Algebra

- Systems of equations: consistency via rank (Rouche-Capelli), unique/infinite/no solution, homogeneous systems, rank-nullity (rank + nullity = number of columns).
- Determinants: properties under row operations, $\det(AB) = \det(A)\det(B)$, $\det(A^T) = \det(A)$, $\det(kA) = k^n \det(A)$.
- Eigenvalues: sum equals trace, product equals determinant; eigenvalues of triangular matrix are its diagonal; A^k has eigenvalues λ^k ; inverse has $1/\lambda$; Cayley-Hamilton theorem; real symmetric matrices have real eigenvalues and orthogonal eigenvectors.

- LU decomposition, rank of a product ($\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$), singular vs invertible matrices, orthogonal and idempotent matrices.

3C. Calculus

- Limits (L'Hopital, standard limits), continuity, differentiability (differentiable implies continuous, not the reverse).
- Mean value theorem, Rolle's theorem, maxima and minima using the first and second derivative tests.
- Definite integral properties (symmetry, even/odd functions), Taylor/Maclaurin series basics.
- Patterns: limit evaluation, find maxima/minima, integral by properties, check continuity or differentiability at a point.

3D. Probability and Statistics

Item	Fact
Core rules	$P(A B) = P(A \text{ and } B)/P(B)$. Total probability and Bayes: $P(A B) = P(B A)P(A)/P(B)$. Independence: $P(A \text{ and } B) = P(A)P(B)$
Expectation and variance	$E[aX+b] = aE[X]+b$; $\text{Var}(aX+b) = a^2\text{Var}(X)$; $\text{Var}(X) = E[X^2] - (E[X])^2$; $\text{Var}(X+Y) = \text{Var } X + \text{Var } Y + 2\text{Cov}(X,Y)$
Binomial(n,p)	Mean np , variance $np(1-p)$
Poisson(lambda)	Mean = variance = λ
Geometric(p)	Mean $1/p$ (number of trials until first success)
Uniform(a,b)	Mean $(a+b)/2$, variance $(b-a)^2/12$
Exponential(lambda)	Mean $1/\lambda$, variance $1/\lambda^2$, memoryless
Normal	Symmetric about the mean; about 68/95/99.7 percent within 1/2/3 standard deviations; sum of independent normals is normal
Standard tools	Markov and Chebyshev inequalities, linearity of expectation (works without independence), indicator random variables

- **Patterns:** conditional/Bayes problems, expected value using indicator variables, dice/coin/card counting, distribution mean-variance questions, hash collision and birthday-style problems.
- **Practise:** 20 to 25 PYQs per sub-topic after finishing each; keep a one-page formula sheet.

4. Digital Logic

Approx. weightage: 5-7 marks (planning estimate from past-paper trends, not official)

A small, mechanical subject where practice speed converts directly to marks. Study before Computer Organization.

Must-know concepts

- **Boolean algebra:** identities, De Morgan, duality, consensus theorem, SOP and POS forms, minterms and maxterms, canonical forms.
- **Minimisation:** K-map (up to 4 variables), prime and essential prime implicants, don't-care conditions, number of literals in minimal form.
- **Combinational circuits:** half and full adders, ripple and carry-lookahead adder delay, subtractors, multiplexers, decoders, encoders, priority encoders, comparators, implementing functions with MUX/decoder/ROM/PLA.
- **Sequential circuits:** SR/JK/D/T latches and flip-flops with characteristic tables and equations, excitation tables, conversion between flip-flops, registers, shift registers, ripple and synchronous counters, mod-N counters, ring and Johnson counters, Mealy vs Moore machines, state minimisation.

- **Number systems:** base conversion, 1's and 2's complement, signed magnitude, BCD, Gray code, overflow detection, IEEE 754 floating point.

Formulas and facts to memorise

Item	Fact
Full adder	Sum = $A \oplus B \oplus C_{in}$; Carry = $AB + BC_{in} + AC_{in}$
2's complement range (n bits)	-2^{n-1} to $2^{n-1} - 1$; negation = invert and add 1
MUX implementation	A function of n variables needs a $2^{n-1}:1$ MUX (use one variable as data input)
Counters	Mod-N needs $\lceil \log_2 N \rceil$ flip-flops. Ring counter: n flip-flops, n states. Johnson (twisted ring): n flip-flops, $2n$ states
Flip-flop equations	D: $Q^+ = D$. T: $Q^+ = T \oplus Q$. JK: $Q^+ = JQ' + K'Q$. SR: $Q^+ = S + R'Q$ ($S=R=1$ invalid)
IEEE 754	Single: 1 sign, 8 exponent (bias 127), 23 mantissa. Double: 1, 11 (bias 1023), 52. Value = $(-1)^s \times 1.m \times 2^{e-bias}$
Gates	NAND and NOR are universal; XOR is odd-parity; XNOR is equality

Frequent question patterns

- Minimal SOP/POS of a given function; count prime implicants; MUX/decoder-based function design; counter sequence and number of states; flip-flop conversion; excitation table design; IEEE 754 encode/decode; range and overflow in 2's complement; propagation delay of adders.

Traps and common mistakes

- Reading 'minimal' as 'smallest gate count' when literals are asked. Forgetting don't-cares can be used as 0 or 1 to enlarge groups.
- Miscounting states of Johnson vs ring counters. Sign errors in 2's complement arithmetic and overflow rules (overflow when carry into MSB differs from carry out).
- IEEE 754: forgetting the hidden leading 1 and the bias; special cases (zero, denormals, infinity, NaN).

How to practise

Solve every PYQ on K-maps and counters by hand; 15 to 20 sequential-circuit state-table problems make the topic automatic.

Your background: bit-level and hardware-adjacent thinking from systems programming makes number representation and counters comfortable; still practise K-maps until they are fast.

5. Computer Organization and Architecture

Approx. weightage: 6-9 marks (planning estimate from past-paper trends, not official)

Numerical and concept-heavy; memory hierarchy and pipelining are the two most reliable question sources.

Must-know concepts

- **Machine instructions and addressing:** instruction formats (0/1/2/3-address), addressing modes (immediate, direct, indirect, register, register indirect, indexed, base-register, relative, auto-increment/decrement), RISC vs CISC, instruction encoding and opcode space.
- **CPU design:** datapath and control unit, hardwired vs microprogrammed control, control store size, horizontal vs vertical microcode, ALU, register files.
- **Pipelining:** stages, structural/data/control hazards, RAW/WAR/WAW, forwarding, stalls, branch prediction and delay slots, speedup and CPI with stalls.
- **Memory hierarchy:** cache mapping (direct, fully associative, set-associative), address breakdown, replacement (LRU, FIFO), write-through/write-back, write-allocate, multilevel caches, AMAT, virtual memory and TLB from the hardware side.

- **I/O:** programmed I/O, interrupt-driven I/O, DMA (burst and cycle stealing), interrupt priorities; secondary storage: disk access time.
- **Performance:** CPU time, MIPS/MFLOPS, Amdahl's law.

Formulas and facts to memorise

Item	Fact
Cache address split	Offset bits = $\log_2(\text{block size})$. Index bits = $\log_2(\text{number of sets})$. Tag = remaining address bits. Sets = cache size / (block size x associativity)
Cache size in bits	Data bits + (tag + valid + dirty bits) per line x number of lines
AMAT	Hit time + miss rate x miss penalty. Two-level: $H_1 + m_1(H_2 + m_2 \times \text{penalty})$
CPU time	Instruction count x CPI x cycle time; CPI with stalls = ideal CPI + stall cycles per instruction
Pipeline speedup	Ideal: k stages. For n instructions: time = (k + n - 1) cycles; speedup = $nk/(k + n - 1)$
Amdahl's law	Speedup = $1 / ((1 - f) + f/s)$
Disk access time	Seek + rotational latency (half a rotation on average) + transfer time
Microcode	Control store = number of microinstructions x width; horizontal = wide, fewer cycles; vertical = narrow, needs decoding
DMA	Cycle stealing: CPU loses one bus cycle per transfer; burst: DMA holds the bus for the block

Frequent question patterns

- Cache: number of tag bits, lines, sets, hits/misses for a reference string, total bits; pipeline: total cycles with hazards and forwarding; CPI/speedup; addressing mode effective-address problems; instruction format encoding (how many instructions fit); microprogram control store size; DMA percentage of CPU time; disk timing; IEEE and arithmetic occasionally.

Traps and common mistakes

- Mixing bits and bytes; forgetting valid/dirty bits in total cache size. Counting index bits for the wrong associativity.
- Pipeline: counting the wrong number of stalls when forwarding exists (a load-use hazard still stalls one cycle).
- Confusing byte-addressable with word-addressable memory in address-width questions.

How to practise

Do 30+ cache and 20+ pipeline problems; write the address-split recipe once and apply it mechanically.

Your background: cache-friendliness and memory layout from performance work give real intuition; the exam still rewards exact bit arithmetic.

6. Programming and Data Structures

Approx. weightage: 6-9 marks (planning estimate from past-paper trends, not official)

C programming is tested as a language of hand-traceable programs, plus classical data structures. Trace by hand; the question is the exact output.

Must-know concepts

- **C basics:** operator precedence and associativity, integer promotion and overflow, implicit conversions, sizeof, bit operators, static/global/local scope and lifetime, storage classes.
- **Pointers and arrays:** pointer arithmetic, pointers to pointers, arrays vs pointers, 2-D arrays and row-major layout, function pointers, strings and null terminators, dangling pointers.
- **Functions and recursion:** call by value, stack frames, tracing recursive calls, tail recursion, output ordering with printf, macros vs functions.
- **Structs and unions:** layout, padding (when stated), unions sharing memory.
- **Data structures:** stacks (expression evaluation, infix-postfix), queues (circular queue conditions, queue using stacks and stacks using queues), linked lists (reversal, insertion, deletion, cycle detection), trees (traversals, reconstruction from two traversals, height, BST insert/delete, AVL rotations, heaps, build-heap), hashing (chaining, linear/quadratic probing, double hashing, load factor), graphs (adjacency matrix/list representation costs).

Formulas and facts to memorise

Item	Fact
Binary tree counts	Distinct binary trees with n nodes: Catalan number $C(2n,n)/(n+1)$. Nodes with 2 children = leaves - 1
Heap	Build-heap is $O(n)$; insertion $O(\log n)$; array indices: children $2i$ and $2i+1$ (1-indexed); leaves start at $\text{floor}(n/2)+1$
Binary tree height	Max nodes at height h : $2^{h+1} - 1$ (root at height 0). Min height with n nodes: $\text{floor}(\log_2 n)$
AVL	Balance factor in $\{-1,0,1\}$; rotations LL, RR, LR, RL; minimum nodes $N(h) = N(h-1) + N(h-2) + 1$
Hashing	Load factor $\alpha = n/m$; chaining expected search $1 + \alpha$; open addressing needs $\alpha < 1$
Circular queue	Full: $(\text{rear} + 1) \bmod n == \text{front}$ (one slot left empty); empty: $\text{front} == \text{rear}$
Traversal reconstruction	Inorder plus either preorder or postorder gives a unique tree; preorder plus postorder alone does not (unless full)
Precedence trap	In C, $i++ + ++i$, $i = i++$ and unsequenced modifications are undefined behaviour - GATE avoids them or states behaviour

Frequent question patterns

- Program output by tracing (loops, recursion, pointers, static variables); stack/queue operations after a sequence; postfix evaluation; tree traversal outputs and reconstruction; BST insertions and deletions and resulting shape; heap after insertions/deletions; hash table contents after insertions; linked list pointer manipulation; number of trees or rotations.

Traps and common mistakes

- Assuming int size incorrectly: read the question, otherwise the usual assumption is 4-byte int on a byte-addressable machine.
- Evaluation order of function arguments and printf arguments is unspecified; follow only what the question fixes.

- Off-by-one in array indexing, forgetting integer division truncation and % with negatives, signed vs unsigned comparisons.
- Static variables retain values across calls; recursion depth and stack order flip outputs.

How to practise

Solve every PYQ on C output by hand without running it; then check. Practise 10 tree-reconstruction and 10 heap/hash problems.

Your background: your C++ background is an advantage, but the exam uses C semantics and tricky pointer/precedence output questions - do not rely on C++ intuition or STL behaviour.

7. Algorithms

Approx. weightage: 8-11 marks (planning estimate from past-paper trends, not official)

The highest-scoring CS subject: analysis, standard algorithms, and identifying which technique fits.

Must-know concepts

- **Asymptotics:** Big-O/Omega/Theta, ordering functions by growth, comparing exponentials and logs, Stirling's approximation.
- **Recurrences:** Master theorem, recursion tree, substitution; $T(n)=aT(n/b)+f(n)$.
- **Sorting and searching:** insertion, selection, bubble, merge, quick, heap, counting, radix, bucket; stability, in-place, best/average/worst cases; comparison-sort lower bound $\Omega(n \log n)$; binary search.
- **Divide and conquer:** merge sort, quicksort partition and pivot choice, Strassen ($n^{\log_2 7}$), closest pair, median of medians, maximum subarray.
- **Greedy:** activity selection, fractional knapsack, Huffman coding, job sequencing with deadlines, Kruskal and Prim; when greedy fails.
- **Dynamic programming:** LCS, edit distance, 0/1 knapsack, matrix chain multiplication, LIS, coin change, Floyd-Warshall, optimal BST.
- **Graph algorithms:** BFS, DFS and classification of edges, topological sort, strongly connected components, MST properties (cut and cycle), shortest paths (Dijkstra, Bellman-Ford, Floyd-Warshall, DAG shortest path), Bellman-Ford negative-cycle detection.
- **Complexity classes:** P, NP, NP-hard, NP-complete; Cook-Levin, reductions (SAT, 3-SAT, clique, vertex cover, independent set, Hamiltonian cycle, subset sum), decision vs optimisation forms.

Formulas and facts to memorise

Item	Fact
Master theorem	$T(n)=aT(n/b)+n^c$: if $c < \log_b a$ then $\Theta(n^{\log_b a})$; if $c = \log_b a$ then $\Theta(n^c \log n)$; if $c > \log_b a$ then $\Theta(n^c)$
Sorting summary	Merge $O(n \log n)$ always, stable, extra space. Quick avg $O(n \log n)$, worst $O(n^2)$, in place, not stable. Heap $O(n \log n)$, in place, not stable. Insertion $O(n)$ best, stable
Graph algorithms	BFS/DFS $O(V+E)$. Dijkstra $O((V+E) \log V)$ with heap; fails with negative edges. Bellman-Ford $O(VE)$. Floyd-Warshall $O(V^3)$. Kruskal $O(E \log E)$. Prim $O(E \log V)$ with heap
Classic DP costs	LCS $O(mn)$. 0/1 knapsack $O(nW)$ pseudo-polynomial. Matrix chain $O(n^3)$. Edit distance $O(mn)$
Huffman	Greedy, $O(n \log n)$; optimal prefix code; cost = sum of frequency x depth
Trees and DFS	DFS timestamps: back edge indicates a cycle in a directed graph; number of topological orders needs DAG

Item	Fact
MST facts	Unique if all edge weights are distinct; every MST contains the lightest edge crossing any cut; MST need not be shortest-path tree
NP-completeness	To show NP-complete: in NP + reduce a known NP-complete problem to it. Reductions go from known problem to new problem
Recurrence shortcuts	$T(n)=T(n-1)+n$ gives $O(n^2)$; $T(n)=2T(n-1)+1$ gives $O(2^n)$; $T(n)=T(n/2)+1$ gives $O(\log n)$; $T(n)=T(\sqrt{n})+1$ gives $O(\log \log n)$

Frequent question patterns

- Solve recurrences (master and non-master); order functions by growth rate; count comparisons/swaps of a sort on a given input; identify time complexity of a code snippet with nested loops; DP table filling and value at a cell; algorithm correctness statements (which are true/false); MST edge weight and number of MSTs; shortest-path distances after k iterations of Bellman-Ford; DFS/BFS order and edge classification; NP-completeness true/false statements.

Traps and common mistakes

- Dijkstra with negative edges; assuming greedy always works (0/1 knapsack needs DP).
- Misreading loop bounds in complexity questions; log base does not matter asymptotically but does matter for exact counts.
- Quicksort worst case depends on pivot rule; on an already-sorted array with first-element pivot it is $O(n^2)$.
- Reversing reduction direction in NP-hardness proofs.

How to practise

Implement each classic algorithm once in the language you like; then solve 40+ PYQs by type. Keep a table of complexities and trace algorithms on small examples.

Your background: regular coding practice on DSA problems directly pays off here; the exam adds analysis and true/false statement questions, so practise proofs of correctness and complexity too.

8. Theory of Computation

Approx. weightage: 6-8 marks (planning estimate from past-paper trends, not official)

Abstract but pattern-driven; the same machine, language and closure questions repeat every year.

Must-know concepts

- Regular languages:** DFA, NFA, epsilon-NFA, subset construction, regular expressions and their equivalence with automata, minimisation, Myhill-Nerode theorem, pumping lemma.
- Context-free languages:** CFG derivations and parse trees, ambiguity, Chomsky and Greibach normal forms, PDA (deterministic and non-deterministic), DCFL vs CFL, pumping lemma for CFL.
- Turing machines:** definition, variants (multi-tape, non-deterministic), recursive vs recursively enumerable languages, Church-Turing thesis, universal Turing machine.
- Decidability:** halting problem, Rice's theorem, decidable problems for regular and CFL classes (emptiness, membership, finiteness, equivalence), reductions.
- Hierarchy:** regular within DCFL within CFL within context-sensitive within decidable within recursively enumerable.

Formulas and facts to memorise

Item	Fact
Subset construction	NFA with n states gives DFA with at most 2^n states
'kth symbol from the end is 1'	Minimal DFA needs 2^k states (NFA needs $k + 1$)

Item	Fact
Closure: regular	Closed under union, intersection, complement, concatenation, Kleene star, reversal, difference, homomorphism, inverse homomorphism
Closure: CFL	Closed under union, concatenation, star, reversal, homomorphism; not closed under intersection, complement; intersection with a regular language is CFL
Closure: DCFL	Closed under complement; not closed under union, intersection, concatenation, star, reversal
Closure: recursive / RE	Recursive: closed under complement, union, intersection. RE: closed under union, intersection, concatenation, star; not complement
Decidability of regular/CFL	Regular: membership, emptiness, finiteness, equivalence all decidable. CFL: membership, emptiness, finiteness decidable; equivalence and universality undecidable
Standard non-regular languages	$a^n b^n$, ww^R (not regular; CFL), ww (not CFL), $a^n b^n c^n$ (not CFL), a^{n^2} , a^p for prime p
Undecidable (Rice)	Any non-trivial property of the language of a Turing machine is undecidable
Machine equivalence	NFA = DFA in power; NPDA is stronger than DPDA; multi-tape and non-deterministic TMs = TM in power

Frequent question patterns

- Number of states of minimal DFA; design a DFA/NFA/PDA for a language; which languages are regular/CFL/decidable; closure property statements; ambiguity of grammars; number of strings of given length accepted; regular expression equivalence; halting/Rice-based true/false; identify the language generated by a CFG.

Traps and common mistakes

- Confusing DPDA-accepted with CFL-accepted classes; DCFL closure is often the subtle one.
- Assuming a language is regular because it looks simple (unbounded counting is never regular). Concatenation and star closure need care for complement questions.
- Not simplifying regular expressions before testing equivalence; overlooking the empty string.

How to practise

Draw the DFA by hand for 25+ languages; keep a closure-property table and a decidability table; solve 40+ PYQs by pattern.

9. Compiler Design

Approx. weightage: 4-6 marks (planning estimate from past-paper trends, not official)

Mechanical and quick to master; most marks come from parsing tables and data-flow analysis.

Must-know concepts

- **Phases:** lexical analysis, syntax analysis, semantic analysis, intermediate code, optimisation, code generation; symbol table and error handling.
- **Lexical analysis:** tokens, patterns, lexemes, longest-match rule, regular expression to DFA, lexical errors.
- **Parsing (top-down):** left recursion elimination, left factoring, FIRST and FOLLOW sets, LL(1) grammar test and parse table, recursive-descent parsing.
- **Parsing (bottom-up):** shift-reduce, handles, LR(0) items, SLR(1), LALR(1), CLR(1), conflicts, operator precedence, ambiguity and how precedence/associativity resolve it.
- **Syntax-directed translation:** synthesised and inherited attributes, S-attributed and L-attributed definitions, dependency graphs.

- **Intermediate code:** three-address code, quadruples, triples, indirect triples, DAG for expressions, basic blocks and flow graph construction.
- **Runtime environments:** activation records, stack vs heap allocation, static and dynamic scoping, parameter passing (value, reference, copy-restore, name).
- **Optimisation:** constant folding, common subexpression elimination, dead code elimination, copy propagation, loop-invariant code motion, strength reduction, register allocation by graph colouring; data-flow analyses: reaching definitions, live variables, available expressions.

Formulas and facts to memorise

Item	Fact
Parser power	$LR(0) < SLR(1) < LALR(1) < CLR(1)$. LL(1) grammars are a subset of LR(1) but not of LR(0) in general
State counts	LR(0), SLR(1) and LALR(1) automata have the same number of states; CLR(1) has more (or equal)
LL(1) conditions	No left recursion, no ambiguity; for $A \rightarrow a \mid b$: FIRST(a) and FIRST(b) disjoint; if epsilon in FIRST(a) then FIRST(b) and FOLLOW(A) disjoint
FIRST/FOLLOW	FOLLOW(start) includes \$. FIRST of terminal = the terminal. Nullable symbols need propagation
Conflicts	Shift-reduce arises from ambiguity/precedence; reduce-reduce from grammar structure. LALR can add reduce-reduce (not shift-reduce) conflicts over CLR
Basic blocks	Leaders: first instruction, target of a jump, instruction after a jump. Block = leader up to next leader
Data-flow direction	Reaching definitions and available expressions: forward. Live variables: backward. Available: intersection (must); reaching/live: union (may)
Parameter passing	Call by name behaves like textual substitution and can give different results from call by reference or value
Three-address code	Quadruples: op, arg1, arg2, result. Triples reference instruction positions; indirect triples use a pointer list allowing reordering

Frequent question patterns

- Compute FIRST/FOLLOW, build LL(1) or SLR table, count states/items; find conflicts; classify a grammar (LL(1), LR(0), SLR(1), LALR(1)); count tokens in a source line; generate three-address code and count temporaries/quadruples; number of basic blocks; data-flow equations; output of a program under different parameter-passing/scoping rules; attribute evaluation; dead-code or loop optimisation applied.

Traps and common mistakes

- Forgetting to add the augmented production in LR items; including epsilon incorrectly in FOLLOW.
- Counting tokens: longest-match rule, and comments/whitespace are not tokens.
- Mixing 'may' and 'must' analyses in data-flow questions.

How to practise

Do 15 FIRST/FOLLOW, 10 LL(1) tables and 10 SLR/CLR/LALR problems by hand; solve all data-flow and three-address PYQs.

Your background: toolchain and build-system familiarity helps with phases and optimisation intuition, but the exam wants exact table and set computation.

10. Operating Systems

Approx. weightage: 7-10 marks (planning estimate from past-paper trends, not official)

Numerical scheduling and paging problems plus conceptual synchronization and deadlock questions.

Must-know concepts

- **Processes and threads:** process states, PCB, context switch cost, user vs kernel threads, thread models, fork/exec/wait behaviour, zombie and orphan processes.
- **CPU scheduling:** FCFS, SJF (non-preemptive), SRTF (preemptive), round robin, priority scheduling, multilevel queue and feedback queue; turnaround, waiting and response time; convoy effect and starvation.
- **Concurrency and synchronization:** race conditions, critical-section requirements (mutual exclusion, progress, bounded waiting), Peterson's algorithm, test-and-set, semaphores (binary and counting), mutexes, monitors, classical problems (producer-consumer, readers-writers, dining philosophers).
- **Deadlock:** four necessary conditions, resource-allocation graph, banker's algorithm (safe sequence), detection and recovery, prevention vs avoidance.
- **Memory management:** contiguous allocation and fragmentation, first/best/worst fit, paging and page tables, multilevel page tables, TLB and effective access time, segmentation.
- **Virtual memory:** demand paging, page fault handling, page replacement (FIFO, LRU, Optimal, Clock), Belady's anomaly, thrashing, working set, copy-on-write.
- **File systems and I/O:** file allocation (contiguous, linked, indexed, inodes), directory structures, free-space management, disk scheduling (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK), disk latency, RAID at concept level, system calls and interrupts.

Formulas and facts to memorise

Item	Fact
Scheduling metrics	Turnaround = completion - arrival. Waiting = turnaround - burst. Response = first run - arrival
Paging basics	Page table entries = virtual address space / page size. Page table size = entries x entry size. Number of frames = physical memory / page size
Multilevel paging	Choose bits so each table fits in one page; levels = $\text{ceil}((\text{VA bits} - \text{offset bits}) / \text{bits resolved per level})$
TLB / EMAT	$\text{EMAT} = h(t + m) + (1 - h)(t + 2m)$ for single-level page table; extend by one memory access per extra level
Demand paging	$\text{EAT} = (1 - p) \times \text{memory access} + p \times \text{page-fault service time}$
fork() counting	n sequential fork() calls create $2^n - 1$ child processes (2^n total). fork returns 0 in the child, child PID in the parent
Banker's algorithm	Safe state has an ordering where each process's need can be satisfied by Available plus resources released by earlier ones; need = max - allocation
Inode file size	Max file size = (direct + single indirect + double indirect + triple indirect) blocks x block size, with pointers per block = block size / pointer size
Page replacement	FIFO can show Belady's anomaly; LRU and Optimal (stack algorithms) do not. Optimal replaces the page used farthest in the future
Disk scheduling	SCAN goes to the end of the disk and reverses; LOOK reverses at the last request; C-SCAN/C-LOOK return without servicing on the way back
Semaphores	Binary semaphore = mutual exclusion. Wrong order of wait operations can deadlock; ordering fixes many dining-philosopher-style questions

Frequent question patterns

- Gantt chart and average waiting/turnaround time; number of processes after `n fork()` calls and printed output; semaphore program: which outputs or deadlocks are possible; is this synchronization solution correct (mutual exclusion/progress/bounded waiting); banker's safe state; page table size and levels; EMAT with TLB and page faults; page faults for a reference string under FIFO/LRU/Optimal; disk head movement; inode file size; system call/interrupt concepts; threads vs processes.

Traps and common mistakes

- Forgetting arrival times in scheduling; mixing preemptive and non-preemptive; round robin ties when a process arrives at the same instant as a quantum expires.
- Assuming FIFO improves with more frames (Belady's anomaly); mixing page and frame counts.
- `fork()` with buffered `printf`: buffered output can duplicate lines; know when the buffer is flushed.
- TLB and page-table accesses: account for the extra memory access on a miss and for page-table memory accesses for multi-level tables.

How to practise

Do 25+ scheduling and 20+ paging/EMAT problems, all fork and semaphore PYQs, and write one banker's algorithm trace.

Your background: everyday Linux system programming (`fork/exec`, signals, threads, pipes, mutexes, virtual memory behaviour) makes this your strongest subject - use the speed to protect time for weaker areas.

11. Databases

Approx. weightage: 6-9 marks (planning estimate from past-paper trends, not official)

Mix of design theory (FDs, normal forms), SQL/relational algebra, transactions and indexing.

Must-know concepts

- **ER model:** entities, attributes, relationships, cardinality and participation constraints, weak entities, generalisation/specialisation, ER to relational mapping and the resulting number of relations.
- **Relational model and algebra:** keys (super, candidate, primary, foreign), select, project, union, set difference, Cartesian product, joins (natural, theta, outer), division, expressing queries, equivalences.
- **Tuple relational calculus:** safe expressions, quantifiers, translating to and from algebra.
- **SQL:** DDL/DML, joins, GROUP BY and HAVING, subqueries and correlated subqueries, EXISTS/IN, aggregates, NULL handling and three-valued logic, views, set operations.
- **Functional dependencies and normalisation:** attribute closure, candidate keys, minimal/canonical cover, 1NF/2NF/3NF/BCNF tests, lossless decomposition test, dependency preservation, multivalued dependencies and 4NF at concept level.
- **Transactions:** ACID, schedules, conflict and view serializability, precedence graph, recoverability (recoverable, cascadeless, strict), locking (two-phase locking and variants, deadlock prevention with wait-die/wound-wait), timestamp ordering, isolation levels.
- **Storage and indexing:** blocking factor, file organisation, dense/sparse and primary/clustering/secondary indexes, multilevel indexing, B-tree and B+ tree (order, height, insertion/deletion), hashing overview.

Formulas and facts to memorise

Item	Fact
Keys and closure	X is a super key iff X^+ contains all attributes. Candidate key: minimal super key. Prime attribute belongs to some candidate key
Normal forms	2NF: no partial dependency of non-prime attributes on a key part. 3NF: for every FD $X \rightarrow A$, X is a super key or A is prime. BCNF: every determinant is a super key

Item	Fact
Decomposition	Lossless join test via chase/table method or $R1 \cap R2$ determines $R1$ or $R2$. BCNF is lossless but may lose dependencies; 3NF always allows lossless and dependency-preserving
Serializability	Conflict serializable iff precedence graph is acyclic. Conflicting: same item, different transactions, at least one write. View serializable is a superset
Recoverability	Strict is inside cascadeless is inside recoverable. 2PL guarantees serializability but not deadlock-freedom or recoverability; strict 2PL ensures strict schedules
Indexing	Blocking factor = $\text{floor}(\text{block size} / \text{record size})$. Sparse index has one entry per block; dense one per record. Blocks for index = $\text{ceil}(\text{entries} / \text{index blocking factor})$
B+ tree	Order p : internal nodes hold up to p pointers and $p - 1$ keys; leaf holds data pointers and is linked to siblings. Height grows only by root split. Minimum fill about half
Join sizes	Natural join size between 0 and $ R S $; with a foreign key in S referencing R , size = $ S $
SQL NULL	Comparisons with NULL are UNKNOWN; NOT IN with a NULL in the subquery returns no rows; aggregates ignore NULL except COUNT(*)

Frequent question patterns

- Find candidate keys and count them; highest normal form; decompose into BCNF/3NF and check lossless/dependency preservation; ER to relational (number of tables); relational algebra/TRC equivalence and query output; SQL query results (joins, group by, having, correlated subquery, NULLs); precedence graph and serializability; recoverable/cascadeless/strict classification; 2PL schedule questions; B+ tree insertions, order, height, index block counts.

Traps and common mistakes

- Treating a determinant as a key; missing that a superkey is enough for BCNF but 3NF also allows prime right-hand sides.
- SQL: HAVING vs WHERE, duplicates with UNION vs UNION ALL, aggregate with NULLs, and the empty-subquery case.
- Serializability: cycles only with conflicting operations between different transactions; commit order matters for recoverability.

How to practise

Solve 20+ candidate-key and normal-form problems, 20 SQL output questions, 15 precedence-graph questions, and 10 B+ tree/indexing calculations.

Your background: SQL exposure gives comfort, but the exam is heavy on formal design theory - practise FD closure and decomposition by hand.

12. Computer Networks

Approx. weightage: 6-9 marks (planning estimate from past-paper trends, not official)

Layered concepts with calculation-heavy topics (data link, subnetting, TCP) and application-layer protocol facts.

Must-know concepts

- **Layers and physical layer:** OSI and TCP/IP models, encapsulation, switching (circuit, packet, message), transmission media, bandwidth, latency components (transmission, propagation, queuing, processing), Nyquist and Shannon capacity, multiplexing.
- **Data link layer:** framing, error detection (parity, checksum, CRC), Hamming code, flow control (stop-and-wait, go-back-N, selective repeat), sliding window efficiency, medium access (ALOHA, slotted ALOHA, CSMA, CSMA/CD, CSMA/CA), Ethernet frame format, minimum frame size, switches, bridges, VLANs.
- **Network layer:** IPv4 addressing and header, classful and classless addressing (CIDR), subnetting/VLSM, fragmentation, NAT, ARP/RARP/DHCP/ICMP, routing (distance vector, link state, count-to-infinity, split horizon), RIP, OSPF, BGP concepts, IPv6 basics.
- **Transport layer:** UDP, TCP segment and header, connection setup/teardown, sequence and acknowledgement numbers, flow control (receiver window), congestion control (slow start, congestion avoidance, fast retransmit/recovery, timeouts), RTT and RTO estimation, sockets.
- **Application layer:** DNS (resolution, caching, record types), HTTP (persistent vs non-persistent, RTT counts, methods), SMTP/POP3/IMAP, FTP, basics of security (encryption, public key, digital signatures, firewalls).

Formulas and facts to memorise

Item	Fact
Bit rate limits	Nyquist (noiseless): $2B \log_2 L$ bits per second. Shannon: $C = B \log_2(1 + \text{SNR})$; SNR from dB: $\text{SNR} = 10^{\text{dB}/10}$
Delay	Transmission = L/R ; propagation = d/v ; total one-way = sum of these plus queuing and processing. $a = T_p/T_t$
Stop-and-wait	Efficiency = $1/(1 + 2a)$
Sliding window	Efficiency = $N/(1 + 2a)$ up to 1. Go-back-N: sender window up to $2^k - 1$, receiver window 1. Selective repeat: window up to 2^{k-1} each side (k = sequence number bits)
CSMA/CD	Frame transmission time must be at least 2 x propagation delay; minimum Ethernet frame 64 bytes; max efficiency approx $1/(1 + 6.44a)$
ALOHA	Pure ALOHA max throughput $1/(2e)$ approx 18.4 percent; slotted ALOHA $1/e$ approx 36.8 percent
Hamming code	Parity bits r satisfy $2^r \geq m + r + 1$. Minimum distance d detects $d - 1$ errors, corrects $\text{floor}((d - 1)/2)$
CRC	Append n zeros for an n -bit-degree generator; remainder is the CRC; divisor x^n polynomial of degree n
IPv4	32-bit address; header 20 to 60 bytes; subnet: hosts = $2^h - 2$; fragment offsets are in units of 8 bytes; total length ≤ 65535
Routing	Distance vector = Bellman-Ford (count-to-infinity problem, fixed by split horizon/poison reverse); link state = Dijkstra on flooded topology
TCP	Handshake SYN, SYN-ACK, ACK; termination via FIN in each direction. Slow start doubles cwnd each RTT until ssthresh; then linear growth; on timeout ssthresh = cwnd/2 and cwnd = 1 MSS; on 3 dup ACKs (fast recovery) ssthresh = cwnd/2 and cwnd resumes at ssthresh
Throughput and window	Max throughput = window size / RTT; sequence-number wrap around time relative to bandwidth

Item	Fact
HTTP timing	Non-persistent: 2 RTT per object (+ transmission); persistent with pipelining: about 1 RTT for all objects after connection

Frequent question patterns

- Delay calculations (packet/message switching); stop-and-wait/sliding-window efficiency and optimal window; minimum frame size or maximum cable length for CSMA/CD; Hamming/CRC computation; subnetting, CIDR aggregation, number of hosts, fragmentation offsets and counts; routing table updates in distance-vector (or Dijkstra); TCP cwnd evolution over RTTs with timeout/dup-ACK events; DNS/HTTP RTT counting; NAT and ARP behaviour; sequence-number bits and window relationship.

Traps and common mistakes

- Using 2^h instead of $2^h - 2$ for usable hosts; mixing bytes and bits and units (Kb vs KB) in delay calculations.
- Forgetting that fragment offsets are in 8-byte units and headers are copied to each fragment.
- TCP: slow start threshold rules differ between timeout and triple duplicate ACK; mixing cwnd in bytes vs MSS.
- Ignoring propagation delay in efficiency calculations or applying a wrong formula to a window-size question.

How to practise

Solve 30+ delay and efficiency problems, 25+ subnetting/fragmentation, 15+ TCP cwnd traces; keep a protocols cheat sheet for application-layer facts.

Your background: socket programming and TCP/IP practical exposure help you with the transport and application layers; data-link efficiency and subnetting still need drill.

13. General Aptitude (15 marks)

Common to every GATE paper: 5 questions of 1 mark and 5 of 2 marks. Target 13 to 15 - it is the most reliable score in the exam.

Area	Topics	Practice focus
Verbal	Grammar (tenses, articles, prepositions, subject-verb agreement), vocabulary (synonyms, antonyms, one-word, idioms), reading comprehension, sentence completion and ordering	1 passage a day; keep a small word list from PYQs
Quantitative	Ratios, percentages, profit and loss, time and work, speed/distance, data interpretation, permutations and combinations, probability, series, geometry and mensuration, logs and powers	10-15 mixed problems a day; speed over depth
Analytical	Deduction, induction, logical reasoning, syllogisms, analogies, seating and arrangement puzzles, numerical relations	Diagrams first; check for unique answer
Spatial	Paper folding and cutting, cubes and dice, mirror images, rotations, pattern completion, 2D/3D assembly	Visualise on paper; there are usually a few per exam

- Do not skip GA on the exam day - solve it in about 20 minutes, mentally on easy ones and on paper for arrangement puzzles.
- The same GA practice serves your DA paper; one habit covers both.

14. 19-week schedule (28 Sep 2026 to 6 Feb 2027)

Assumes about 20 to 25 hours per week alongside a full-time job. Each week: topic study on weekdays (about 2 hours), a longer block on Saturday, review and a test on Sunday. Because you are also taking DA, the Maths weeks double as DA preparation; the C++ roadmap goes to a light maintenance slot until the exam.

Weeks	Dates	Focus	Deliverable
1-2	28 Sep - 11 Oct	Setup: diagnostic on all CS subjects, official syllabus, PYQs collected, one book per subject; start GA daily; Discrete Maths part 1 (logic, sets, relations)	Baseline scores, weak list, fixed timetable
3-4	12 Oct - 25 Oct	Discrete Maths part 2 (functions, groups, graphs, combinatorics); Linear Algebra; PYQs	Maths formula sheet started
5-6	26 Oct - 8 Nov	Probability and Statistics; Calculus; Algorithms part 1 (asymptotics, recurrences, sorting, D&C;)	Maths sectional test; Algorithms table
7-8	9 Nov - 22 Nov	Algorithms part 2 (greedy, DP, graph algorithms, NP); Programming and Data Structures (C tracing, stacks, queues, lists)	Algorithms and PDS sectionals
9-10	23 Nov - 6 Dec	PDS part 2 (trees, heaps, hashing, graphs); Operating Systems part 1 (processes, scheduling, synchronization)	Scheduling and sync PYQs done
11-12	7 Dec - 20 Dec	Operating Systems part 2 (deadlock, memory, virtual memory, files); Databases part 1 (ER, algebra, SQL, FDs, normal forms)	OS sectional; FD and NF drill
13	21 Dec - 27 Dec	Databases part 2 (transactions, indexing/B+); Computer Networks part 1 (layers, data link)	DBMS sectional
14	28 Dec - 3 Jan	Computer Networks part 2 (network, transport, application); Digital Logic	CN and DL sectionals
15	4 Jan - 10 Jan	Computer Organization (pipelining, cache, I/O); Theory of Computation part 1	COA sectional; TOC tables
16	11 Jan - 17 Jan	Theory of Computation part 2; Compiler Design; first full-length mocks (CS)	Every topic touched once; error log active
17	18 Jan - 24 Jan	Full mocks alternating CS and DA (2 per week); full PYQ papers; revise weak topics from the error log	Time strategy fixed
18-19	25 Jan - 6 Feb	Taper: formula-sheet revision, one timed CS mock, admit-card checks, sleep and routine	Calm exam day

If the schedule slips, drop breadth before you drop mocks: it is better to know 85 percent of the syllabus deeply and have three full mocks analysed than to cover 100 percent shallowly. If your CS slot in February comes earlier than 6 Feb, pull the taper forward - do not compress the mock weeks.

15. PYQs, mocks, error log and revision system

PYQ method

- First pass: topic-wise PYQs straight after finishing a topic, untimed, with full analysis of every wrong or guessed answer.
- Second pass: mixed PYQs by subject, timed at about 2.5 minutes per question.
- Third pass: full past papers under exam conditions in the last 6 weeks.
- Cover roughly the last 10 to 15 years; older papers help for TOC, Algorithms and Discrete Maths fundamentals.

Error log

Column	What to record
Question source	Paper/mock and question number
Topic	Sub-topic, not just the subject
Error type	Concept gap, formula slip, calculation, misread, time pressure, guessed wrong
Fix	The rule or trick that would have worked
Re-test date	Redo the question after 7 days and again after 30 days

Mock analysis (per mock)

- Score by section, attempt rate, accuracy, time spent on NAT vs MCQ and on wrong answers.
- Sort mistakes into knowledge, speed and carelessness; fix knowledge first, then speed, then carelessness with a final-check routine.
- Take about 8 to 10 CS mocks in total, the last 4 to 5 under exact exam timing and on your exam-time slot if possible.

Revision cycle

- Weekly Sunday review of the week's topics and error-log items; monthly one-page formula sheets per subject.
- Last 2 weeks: formula sheets, closure/decidability tables (TOC), FIRST/FOLLOW recipes (Compiler), cache and pipeline recipes (COA), fork/semaphore/scheduling recipes (OS).

16. Exam-day strategy, resources and sources

Stage	Time	What to do
General Aptitude	About 20 min	Bank all 15 marks quickly and accurately
First pass, core	About 100 min	Answer everything doable in under 3 minutes; mark long/uncertain ones; attempt NAT/MSQ you are sure about
Second pass	About 45 min	Return to marked questions; eliminate options on MCQs; never leave NAT/MSQ blank
Final review	About 15 min	Check NAT rounding, units, MSQ completeness; re-read any question flagged as ambiguous

- Use the on-screen calculator only when arithmetic is heavy; do many power-of-two and unit conversions mentally.
- Note NAT format (rounding/range) before typing; write intermediate steps on the rough sheet.
- Do not spend more than about 4 minutes on any single 2-mark question in the first pass.
- Sleep well the night before; check ID, admit card, route and centre timing the day before.

Reference books (pick one per subject)

Subject	Common references
Algorithms and DS	Cormen et al. (CLRS); Sahni-Horowitz; Weiss
Operating Systems	Silberschatz-Galvin-Gagne; Tanenbaum
Computer Networks	Kurose-Ross; Forouzan; Tanenbaum
Databases	Silberschatz-Korth-Sudarshan; Ramakrishnan-Gehrke
Theory of Computation	Sipser; Hopcroft-Motwani-Ullman
Compiler Design	Aho-Lam-Sethi-Ullman (Dragon Book)
COA and Digital Logic	Hamacher; Patterson-Hennessy; Morris Mano

Subject	Common references
Discrete Maths	Kenneth Rosen; Liu
C Programming	Kernighan-Ritchie; Kanetkar for practice

Sources and verification. Built from the official GATE CS syllabus as reproduced by GeeksforGeeks and other coverage of the IIT Madras notification, plus standard textbook material. Weightage ranges, timing splits and week-by-week plans are planning estimates, not official figures. Confirm the syllabus, dates, fees and rules on gate2027.iitm.ac.in.